

Transfer Learning, Fine-Tuning, and Representation Reuse

Mathematics of Deep Learning — Chapter 7

Yin Yang

Foundation Books Project

Roadmap

From Scratch vs. Reusing a Model

Source and Target Domains and Tasks

Backbone and Head

Frozen Features and Linear Probing

Replacing and Training the Head

Fine-Tuning from a Pretrained Initialization

Which Layers Transfer?

Representation Diagnostics

Regularization, Data Size, Overfitting

Summary

From Scratch vs. Reusing a Model

The Central Decomposition

The chapter rests on one factorization of a model:

$$f_{\theta,\psi}(x) = h_{\psi}(\phi_{\theta}(x)).$$

- $\phi_{\theta} : \mathcal{X} \rightarrow \mathbb{R}^d$: feature-producing *backbone*.
- $h_{\psi} : \mathbb{R}^d \rightarrow \mathbb{R}^K$: task-specific *head*.
- θ_0 : pretrained value for the backbone parameters.

Target dataset: $\mathcal{D}_T = \{(x_i, y_i)\}_{i=1}^m$, $y_i \in \{1, \dots, 5\}$.

Transfer changes which coordinates of (θ, ψ) are fixed, initialized at θ_0 , or optimized.

Four Feasible Sets, One ERM

Training from scratch optimizes everything from random ω_0 :

$$\min_{\omega} \hat{R}_T(\omega) = \min_{\omega} \frac{1}{m} \sum_{i=1}^m \ell(g_{\omega}(x_i), y_i).$$

Transfer changes the initialization *and* the trainable set:

scratch	random ω , optimize all
frozen features	fix $\theta = \theta_0$, train ψ
partial FT	train ψ and a subset of θ
full FT	train ψ and all of θ

The mathematical question: which feasible set gives the lowest held-out target risk?

When Transfer Helps – and When It Doesn't

Transfer is helpful only relative to a baseline and a validation rule.

- If the source representation captures evidence the target needs: reuse can lower data requirements drastically.
- If not: source initialization adds bias without reducing held-out risk (*negative transfer*).

Transfer gain relative to a no-transfer baseline M_0 :

$$G(M_{\text{tr}}; M_0) = \hat{R}_{\text{val}}(M_0) - \hat{R}_{\text{val}}(M_{\text{tr}}).$$

Positive G is evidence; negative G is evidence of negative transfer relative to that baseline.

Source and Target Domains and Tasks

Domain vs. Task

Domain

Input space plus a probability distribution over inputs.

Task

The prediction problem attached to a domain: label space and the rule being learned. In supervised classification, a joint distribution $\mathbb{P}(x, y)$.

Source problem: $P_S(x, y)$, dataset $\mathcal{D}_S$, label space $\mathcal{Y}_S$.

Target problem: $P_T(x, y)$, dataset $\mathcal{D}_T$, label space $\mathcal{Y}_T$.

Transfer is easiest when the representation-relevant parts of P_S and P_T overlap, even when $\mathcal{Y}_S \neq \mathcal{Y}_T$.

Two Named Shifts

When source and target share input and label semantics:

Covariate shift

$P_S(x) \neq P_T(x)$ while $P_S(y | x) = P_T(y | x)$.

The inputs look different; the labelling rule is the same.

Label shift

$P_S(y) \neq P_T(y)$ while $P_S(x | y) = P_T(x | y)$.

The class frequencies change; class-conditional appearance is fixed.

Naming the shift focuses the diagnosis: re-weight features, re-balance labels, or change the representation.

A Transfer-Risk Ledger (Ben-David et al.)

For every $h \in \mathcal{H}$,

$$\epsilon_T(h) \leq \epsilon_S(h) + \frac{1}{2}d_{\mathcal{H}\Delta\mathcal{H}}(P_S^X, P_T^X) + \lambda_{\mathcal{H}},$$

where

$$\lambda_{\mathcal{H}} = \min_{h \in \mathcal{H}} \{\epsilon_S(h) + \epsilon_T(h)\}.$$

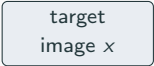
Three quantities to budget:

- $\epsilon_S(h)$: source error.
- $d_{\mathcal{H}\Delta\mathcal{H}}$: how visibly different the inputs look to the class.
- $\lambda_{\mathcal{H}}$: does *any* hypothesis work well on both?

Low source error alone is not enough — the divergence and the joint-error term can each ruin transfer.

Backbone and Head

Splitting a Network – Build It Up



target
image x

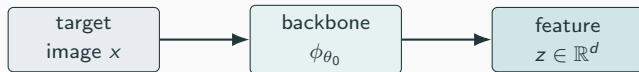
(1) A target image enters the pipeline.

Splitting a Network – Build It Up



- (1) A target image enters the pipeline.
- (2) The pretrained backbone supplies features.

Splitting a Network – Build It Up



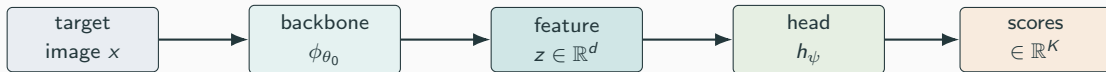
- (1) A target image enters the pipeline.
- (2) The pretrained backbone supplies features.
- (3) Feature vector z lives in $\mathbb{R}^d$.

Splitting a Network – Build It Up



- (1) A target image enters the pipeline.
- (2) The pretrained backbone supplies features.
- (3) Feature vector z lives in $\mathbb{R}^d$.
- (4) A target-specific head reads off scores.

Splitting a Network – Build It Up



- (1) A target image enters the pipeline.
- (2) The pretrained backbone supplies features.
- (3) Feature vector z lives in $\mathbb{R}^d$.
- (4) A target-specific head reads off scores.
- (5) Output dimension K matches the target labels.

Reuse and adaptation live on different boxes: θ on the backbone, ψ on the head.

The Linear Target Head

Replace the source head by a fresh linear target head:

$$h_\psi(z) = Wz + b, \quad W \in \mathbb{R}^{K \times d}, \quad b \in \mathbb{R}^K.$$

Composition $h_\psi(\phi_\theta(x))$ requires the shape conditions

$$\phi_\theta(x) \in \mathbb{R}^d, \quad W \in \mathbb{R}^{K \times d}, \quad K = K_T.$$

Example: $d = 512$, $K_T = 5$ gives $W \in \mathbb{R}^{5 \times 512}$ and $b \in \mathbb{R}^5$, a total of

$$5 \cdot 512 + 5 = 2565 \text{ parameters.}$$

Shape check is part of the model definition — a mismatch means the composition is not a valid function.

Shared Representation, Cheaper Heads

Suppose t related source tasks share representation $h \in \mathcal{H}$:

$$x \mapsto f_j(h(x)), \quad j = 1, \dots, t.$$

Schematic sample-complexity behaviour:

$$\text{cost} \approx C(\mathcal{H}) + t C(\mathcal{F}).$$

- $C(\mathcal{H})$ paid *once* across source tasks.
- $C(\mathcal{F})$ paid *per task*: a new head on existing features.

Relatedness and diversity assumptions do real work; not every pretrained backbone transfers to every target.

Pretraining amortizes representation cost so a new target task mostly pays for its own head.

Frozen Features and Linear Probing

Freezing the Backbone

In a frozen-feature model, $\theta = \theta_0$ is held fixed, only ψ moves:

$$\min_{\psi} \frac{1}{m} \sum_{i=1}^m \ell(h_{\psi}(\phi_{\theta_0}(x_i)), y_i).$$

Define $z_i = \phi_{\theta_0}(x_i)$. The objective is

$$\min_{\psi} \frac{1}{m} \sum_{i=1}^m \ell(h_{\psi}(z_i), y_i),$$

ordinary ERM on the transformed dataset $\{(z_i, y_i)\}_{i=1}^m$.

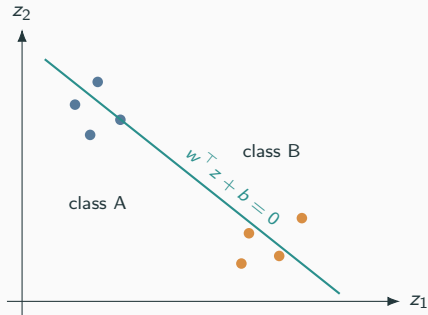
Features can be precomputed once; target training becomes classical classification on fixed vectors.

Linear Probe Geometry – Step by Step



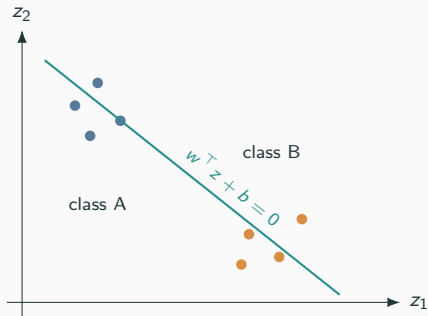
(1) Fixed features z_i landed in $\mathbb{R}^d$.

Linear Probe Geometry – Step by Step



- (1) Fixed features z_i landed in $\mathbb{R}^d$.
- (2) A linear probe can only move an affine boundary.

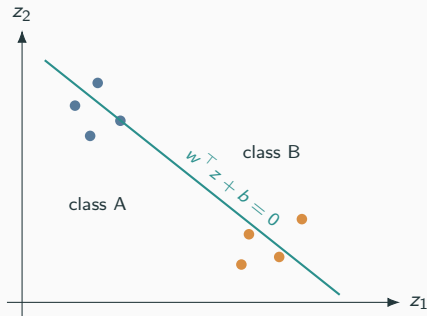
Linear Probe Geometry – Step by Step



Binary probe: $s(z) = w^T z + b$
Region $\{z : w^T z + b \geq 0\}$ is a half-space.

- (1) Fixed features z_i landed in $\mathbb{R}^d$.
- (2) A linear probe can only move an affine boundary.
- (3) Binary case: half-spaces decide.

Linear Probe Geometry – Step by Step



Binary probe: $s(z) = w^T z + b$
Region $\{z : w^T z + b \geq 0\}$ is a half-space.

Multiclass boundary (k vs. ℓ):
 $(W_{k,:} - W_{\ell,:})z + (b_k - b_\ell) = 0$.

- (1) Fixed features z_i landed in $\mathbb{R}^d$.
- (2) A linear probe can only move an affine boundary.
- (3) Binary case: half-spaces decide.
- (4) Multiclass case: pairwise affine hyperplanes.

If features are not linearly separable, the backbone itself must adapt — no probe will save it.

Probe as Baseline and Diagnostic

Linear probe

A linear classifier trained on $z_i = \phi_{\theta_0}(x_i)$ to test how useful the representation already is for the target task.

Reading the probe:

- Good probe accuracy $\Rightarrow$ target classes are already close to linearly separable in the features.
- Poor probe accuracy $\Rightarrow$ need a different head, more data, or to fine-tune the backbone.

Concrete probe: $s(z) = 2z_1 + z_2 - 3$. At $z = (2, 1)$: $s = 2$ (positive). At $z = (0, 1)$: $s = -2$ (negative).

The probe is both a strong baseline and a measurement of feature quality.

Replacing and Training the Head

Why a New Head?

Source head is tied to $\mathcal{Y}_S$:

$$h_{\psi_S}(z) = W_S z + b_S, \quad W_S \in \mathbb{R}^{K_S \times d}.$$

Target head is tied to $\mathcal{Y}_T$:

$$h_{\psi_T}(z) = W_T z + b_T, \quad W_T \in \mathbb{R}^{K_T \times d}.$$

In the running example, $K_S = 1000$ and $K_T = 5$, so the number of output scores changes. A *logit* is one raw score before softmax produces class probabilities. Parameter count, $d = 2048$, $K_T = 5$:

$$5 \cdot 2048 + 5 = 10245.$$

If $\mathcal{Y}_S \neq \mathcal{Y}_T$, the last affine map must be rebuilt before any target loss is meaningful.

Head-Only Warmup

A common conservative schedule:

1. Freeze the backbone at θ_0 .
2. Train the new head until it maps features sensibly.
3. Then unfreeze late backbone layers.

Why? A randomly initialized head produces large gradients $\nabla_{\theta}\ell$ that immediately reshape a good backbone — often destroying useful pretrained structure.

- Not a theorem: small learning rates, warmup, or layer-specific η can also work.
- It is a low-risk default.

Warmup separates “head doesn’t fit target labels yet” from “backbone needs to adapt”.

Fine-Tuning from a Pretrained Initialization

Full fine-tuning makes the backbone trainable. The target empirical risk becomes

$$\hat{R}_T(\theta, \psi) = \frac{1}{m} \sum_{i=1}^m \ell(h_\psi(\phi_\theta(x_i)), y_i).$$

Key differences from training-from-scratch ERM:

- Initialization at $\theta = \theta_0$ (not random).
- Local optimization path from that initialization.
- Often an additional anchor regularizer $\|\theta - \theta_0\|_2^2$.

Once θ moves, the features $z_i = \phi_\theta(x_i)$ change during training.

Fine-tuning lets the representation *move*, which is both its power and its risk.

Gradient Paths: Frozen vs. Fine-Tuned

For one example, write $z = \phi_\theta(x)$ and $\ell = \ell(h_\psi(z), y)$. Let $a = \nabla_z \ell(h_\psi(z), y)$.

Frozen features ($\theta = \theta_0$ fixed)

$$\nabla_\theta \ell = 0, \quad \nabla_\psi \ell \neq 0.$$

Only the head moves.

Full fine-tuning (θ trainable)

$$\nabla_\theta \ell = J_{\phi, \theta}(x)^\top a, \quad \nabla_\psi \ell \neq 0.$$

Same upstream gradient now propagates through the backbone.

Fine-tuning opens gradient paths that move the feature map — not just more weights, but a different kind of update.

Trainable Subsets: Frozen, Partial, Full

frozen
features

early

middle

late

head

few moving parameters

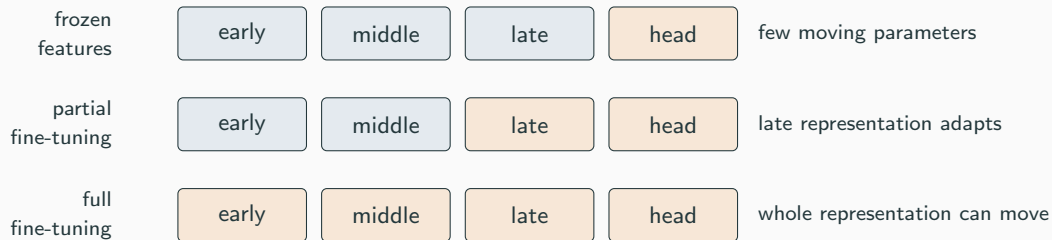
(1) Only head is trainable; cheapest baseline.

Trainable Subsets: Frozen, Partial, Full



- (1) Only head is trainable; cheapest baseline.
- (2) Late blocks unfrozen; source-specific layers adapt.

Trainable Subsets: Frozen, Partial, Full



- (1) Only head is trainable; cheapest baseline.
- (2) Late blocks unfrozen; source-specific layers adapt.
- (3) Everything moves; maximum flexibility *and* risk.

The model class grows monotonically; validation risk decides whether extra flexibility pays.

The Pretrained-Anchor Penalty

Keep θ_0 visible in the optimization:

$$\min_{\theta, \psi} \hat{R}_T(\theta, \psi) + \lambda \|\theta - \theta_0\|_2^2.$$

Gradient of the penalty:

$$\nabla_{\theta} P(\theta) = 2\lambda(\theta - \theta_0).$$

This pulls θ back toward θ_0 — *not* toward zero. Ordinary weight decay $\lambda\|\theta\|_2^2$ would do the latter. One-parameter example: $\theta_0 = 3$, $\lambda = 1$, current $\theta = 3.4$, target gradient -4 , $\eta = 0.1$:

$$g_{\text{total}} = -4 + 2(1)(0.4) = -3.2, \quad \theta \leftarrow 3.4 - 0.1(-3.2) = 3.72.$$

The anchor does not forbid adaptation — it slows movement away from the pretrained value.

Which Layers Transfer?

Layer-wise Representations

Write a depth- L backbone as a composition:

$$\phi_{\theta} = \phi_{\theta_L}^{(L)} \circ \phi_{\theta_{L-1}}^{(L-1)} \circ \dots \circ \phi_{\theta_1}^{(1)}.$$

Representation after layer j :

$$z^{(j)}(x) = \phi_{\theta_j}^{(j)} \circ \dots \circ \phi_{\theta_1}^{(1)}(x).$$

For target distribution q_{tgt} and head class $\mathcal{G}_j$, population probe transferability:

$$T_j(\mathcal{G}_j) = \inf_{g \in \mathcal{G}_j} \mathbb{E}_{(X, Y) \sim q_{\text{tgt}}} \left[\ell(g(z^{(j)}(X)), Y) \right].$$

Lower $T_j \Rightarrow$ layer j is more transferable.

Transferability is relative — a feature can be great for one target family and poor for another.

Choosing the Unfreezing Depth

Frozen probe at depth j :

$$\min_{\psi} \frac{1}{m} \sum_{i=1}^m \ell(h_{\psi}(z_i^{(j)}), y_i).$$

Unfreezing the suffix after layer j :

$$\min_{\theta_{j+1}, \dots, \theta_L, \psi} \frac{1}{m} \sum_{i=1}^m \ell(h_{\psi}(\phi_{\theta_L}^{(L)} \circ \dots \circ \phi_{\theta_{j+1}}^{(j+1)}(z_i^{(j)})), y_i).$$

Choosing a depth = choosing a feasible set: which prefix is fixed, which suffix adapts.

If the late representation is too source-specific, the right fix is often to make the suffix trainable.

General vs. Task-Specific Features

Common empirical pattern in CNNs (Yosinski et al.):

- Early layers: edges, oriented filters → broadly useful.
- Middle layers: textures, parts → moderately reusable.
- Late layers: source-class motifs → task-specific.

General vs. task-specific

A *general* feature is useful across many related tasks. A *task-specific* feature is useful mainly for the source task that shaped it. Both are relative categories.

Co-adaptation: features in different layers learn to work *together*. Splitting at an awkward layer breaks pairs that were jointly tuned.

The right split point usually trades general feature reuse against task-specific suffix adaptation.

Representation Diagnostics

Treat Features as Data

For each target example, compute $z_i = \phi_\theta(x_i) \in \mathbb{R}^d$. Then ask: are the target labels simple in this feature space?

Cosine similarity

For nonzero $u, v \in \mathbb{R}^d$:

$$\text{CosSim}(u, v) = \frac{u^\top v}{\|u\|_2 \|v\|_2}.$$

Measures directional alignment, not magnitude.

Quick check, $z_1 = (1, 1)$, $z_2 = (2, 2)$, $z_3 = (1, -1)$:

$$\text{CosSim}(z_1, z_2) = 1, \quad \text{CosSim}(z_1, z_3) = 0.$$

The diagnostics in this section turn “does the head see good geometry?” into numbers.

Centroids, Scatter, and Separation

Class centroid for target class k , $\mathcal{D}_k = \{i : y_i = k\}$:

$$\mu_k(\theta) = \frac{1}{|\mathcal{D}_k|} \sum_{i \in \mathcal{D}_k} \phi_\theta(x_i).$$

Within-class scatter:

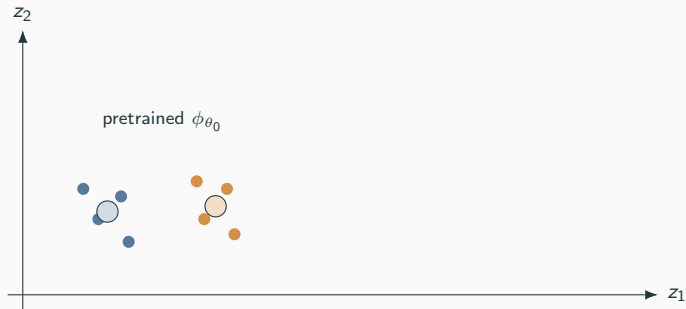
$$S_{\text{within}}(\theta) = \sum_k \sum_{i \in \mathcal{D}_k} \|\phi_\theta(x_i) - \mu_k(\theta)\|_2^2.$$

Between-class separation:

$$S_{\text{between}}(\theta) = \sum_{k < \ell} \|\mu_k(\theta) - \mu_\ell(\theta)\|_2^2.$$

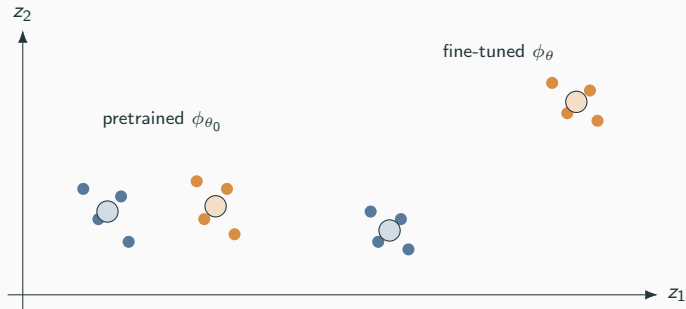
Large S_{between} and small S_{within} $\Rightarrow$ an easy geometry for a simple head.

Representation Drift – Building the Picture



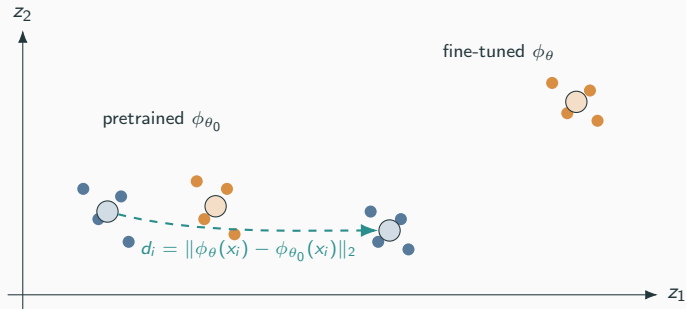
(1) Classes overlap under the pretrained backbone.

Representation Drift – Building the Picture



- (1) Classes overlap under the pretrained backbone.
- (2) After fine-tuning, the centroids pull apart.

Representation Drift – Building the Picture



- (1) Classes overlap under the pretrained backbone.
- (2) After fine-tuning, the centroids pull apart.
- (3) Per-example drift d_i measures how far each feature has moved.

Drift alone is neither good nor bad — pair it with held-out validation evidence.

NN Consistency and CKA

Nearest-neighbor consistency

For $N(i) = \arg \max_{j \neq i} s(\phi_\theta(x_i), \phi_\theta(x_j))$:

$$\frac{1}{m} \sum_{i=1}^m \mathbf{1}\{y_{N(i)} = y_i\}.$$

Fraction of points whose closest neighbour shares its label.

Linear centered-kernel alignment (CKA)

For centered feature matrices $Z, Z_0 \in \mathbb{R}^{m \times d}$:

$$\text{CKA}(Z, Z_0) = \frac{\|Z^\top Z_0\|_F^2}{\|Z^\top Z\|_F \|Z_0^\top Z_0\|_F}.$$

Invariant to orthogonal rotations.

Low CKA + better validation $\Rightarrow$ useful adaptation; low CKA + widening gap $\Rightarrow$ destructive drift.

Regularization, Data Size, Overfitting

Bigger Trainable Set $\neq$ Lower Validation Risk

Let $A \subseteq B$ be trainable subsets with the same frozen initialization. Then on the training objective alone,

$$\inf_{\omega \in \mathcal{F}_B} \hat{R}_T(\omega) \leq \inf_{\omega \in \mathcal{F}_A} \hat{R}_T(\omega).$$

Why? $\mathcal{F}_A \subseteq \mathcal{F}_B$. But this comparison says *nothing* about validation risk:

- Extra flexibility can improve training fit.
- Extra flexibility can simultaneously *widen* the empirical generalization gap.

Freezing reduces variance by shrinking the set of parameter moves the target data can make.

Data Augmentation as Distribution

Let $\mathcal{A}_i$ be a distribution over allowed transformations of x_i . The augmented empirical risk:

$$\hat{R}_{T,\text{aug}}(\theta, \psi) = \frac{1}{m} \sum_{i=1}^m \mathbb{E}_{\tau \sim \mathcal{A}_i} [\ell(f_{\theta, \psi}(\tau(x_i)), y_i)].$$

Label-preservation assumption: for the intended rule y^* ,

$$y^*(\tau(x_i)) = y_i \quad \text{whenever} \quad \mathbb{P}_{\mathcal{A}_i}[\tau] > 0.$$

Examples: horizontal flip $(\tau_{\text{flip}}X)_{i,j,c} = X_{i,W-1-j,c}$, brightness shift $\tau_b(X) = \text{clip}(X + b, 0, 1)$.

Augmentation only works when the transformations preserve the label — the assumption is mathematical, not cosmetic.

Ablating the Trainable Set – A Numerical Picture

Run	Trainable params	Train acc.	Val. acc.
Frozen head	10,245	84%	82%
Final stage + head	2,010,245	90%	87%
Full backbone + head	12,010,245	99%	80%

- Full backbone wins on train, loses on validation.
- Final stage wins where it counts.
- Frozen head is competitive given how few parameters move.

One mathematical principle: compare nested feasible sets by held-out risk, not by training fit.

The Decision Guide

Nested feasible sets, ordered by flexibility:

$$\mathcal{F}_{\text{head}} \subseteq \mathcal{F}_{\text{late}} \subseteq \mathcal{F}_{\text{full}}.$$

Pick by validation risk; modulate with:

- Anchor regularizer $\lambda \|\theta - \theta_0\|_2^2$.
- Layer-specific learning rates (smaller for early layers).
- Label-preserving augmentation.
- Early stopping on a held-out set.

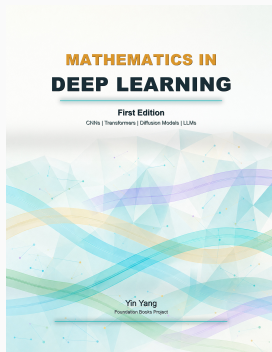
Caveat: a positive transfer gain is relative to a baseline, a split, and a selection rule. If deployment distribution shifts further, the gain need not persist.

Transfer math gives evidence and tradeoffs, not a guarantee that pretraining must help.

Summary

- **Decomposition:** $f_{\theta,\psi}(x) = h_{\psi}(\phi_{\theta}(x))$. Reuse changes the feasible set, not the function-composition view.
- **Source vs. target:** domain, task, covariate vs. label shift, and a transfer-risk ledger $\epsilon_T \leq \epsilon_S + d_{\mathcal{H}\Delta\mathcal{H}}/2 + \lambda_{\mathcal{H}}$.
- **Backbone/head split:** shape conditions d, K make the composition valid; the head usually must be rebuilt.
- **Frozen features:** target training becomes ERM on $\{(z_i, y_i)\}$; a linear probe is both baseline and diagnostic.
- **Fine-tuning:** opens gradient paths $\nabla_{\theta}\ell$ that move the representation; anchor regularizer $\lambda\|\theta - \theta_0\|_2^2$ controls drift.
- **Layer choice:** early/general vs. late/task-specific; pick the unfreezing depth by validation.
- **Diagnostics:** centroids, within/between scatter, drift, NN consistency, CKA — read with validation risk.
- **Selection:** $\mathcal{F}_{\text{head}} \subseteq \mathcal{F}_{\text{late}} \subseteq \mathcal{F}_{\text{full}}$; choose by held-out, not training fit.

Next: regularization techniques as constraints on the same kind of feasible-set thinking that governs transfer.



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/mathematics-in-deep-learning
Code	github.com/foundation-books/mathematics-in-deep-learning-code
Paperback	amazon.com/dp/BOH1ZZHDT2
Hardcover	amazon.com/dp/BOH1ZL11MJ
Kindle	amazon.com/dp/B0GX32M8SP
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.